Python Microservice Project

Inventory & Payment REST APIs with Redis Event Bus

Author	**Hermann**
Date	**May 28, 2026**
Course / Subject	**Microservices Architecture**
Technology Stack	**FastAPI · Redis · React**

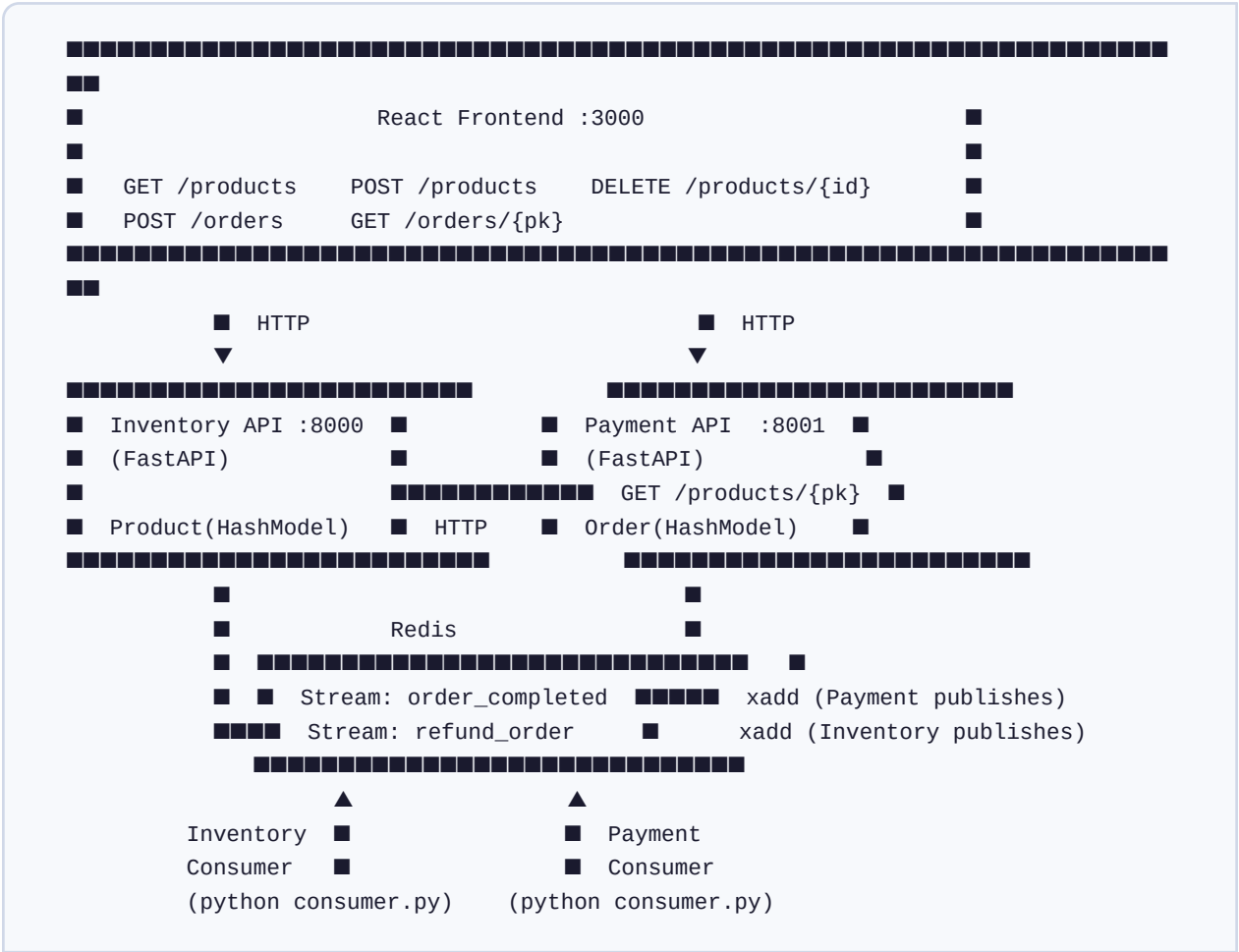
1. Project Overview

This project implements a microservice architecture composed of two independent backend services and a React frontend. Each service owns its data and communicates asynchronously using **Redis Streams** — a message-bus pattern that decouples services so that a failure in one does not block the other.

Services at a glance

Service	Port	Responsibility	Storage
Inventory API	8000	Manage products (create, list, delete, query). Owns product stock levels.	Redis (HashModel)
Payment API	8001	Accept orders, calculate fees, simulate payment, publish completion events.	Redis (HashModel)
React Frontend	3000	Browse products, place orders, auto-poll order status.	—

Architecture Diagram



2. Inventory API (port 8000)

The Inventory service manages the product catalogue. Products are stored as `HashModel` objects in Redis. Every product is automatically assigned a unique `pk` (primary key) by redis-om.

Data Model — Product

```
class Product(HashModel):
    name:     str
    price:    float
    quantity: int

    class Meta:
        database = redis    # shared Redis connection
```

Endpoints

Method	Path	Description	Returns
GET	`/products`	Return all products as a list with `id` field	Array of formatted products
GET	`/products/{pk}`	Return a single product by primary key	Single formatted product
POST	`/products`	Create a new product. Body: `{ name, price, quantity }`	Created product object
DELETE	`/products/{pk}`	Delete a product by primary key	Deleted product object

Example — Create a Product

Request:

```
POST http://localhost:8000/products
Content-Type: application/json

{
  "name":     "orange",
  "price":    20,
  "quantity": 100
}
```

Response 200 OK:

```
{
  "pk":       "01KSQ0KZ57YPWSD8E7GHAA20Z3",
  "name":     "orange",
```

```
  "price":    20.0,
  "quantity": 100
}
```

3. Payment API (port 8001)

The Payment service accepts orders from the frontend, retrieves the current product price from the Inventory service (service-to-service HTTP call), computes the 20% processing fee, persists the order, then runs the payment simulation in a **background task** so that the HTTP response returns immediately.

Data Model — Order

```
class Order(HashModel):
    product_id: str
    price:      float   # unit price from Inventory
    fee:        float   # 20% of price
    total:      float   # price + fee
    quantity:   int
    status:     str     # pending | completed | refunded

    class Meta:
        database = redis
```

Endpoints

Method	Path	Description	Returns
POST	`/orders`	Place a new order. Body: `{ id, quantity }` where `id` is the product `pk`	Order object with `status: "pending"`
GET	`/orders/{pk}`	Retrieve an order's current status. The frontend polls this endpoint every 3 seconds until the status changes from pending.	Order object with current status

How a POST /orders request is processed

```
POST http://localhost:8001/orders
Content-Type: application/json

{
  "id":       "01KSQ0KZ57YPWSD8E7GHAA20Z3",  // product pk
  "quantity": 2
}
```

Response 200 OK (immediate, before payment completes):

```
{
  "pk":         "01KSPZWMF9T3SZP0RQVBN4X8Y2",
  "product_id": "01KSQ0KZ57YPWSD8E7GHAA20Z3",
  "price":      20.0,
```

```
  "fee":        4.0,    // 20% of 20
  "total":      24.0,   // price + fee
  "quantity":   2,
  "status":     "pending"
}
```

4. Redis Streams — Asynchronous Event Bus

Redis Streams act as a durable message queue between services. Unlike simple pub/sub, streams persist messages and track delivery with **consumer groups**, guaranteeing that every message is processed exactly once. If a consumer crashes, the message stays in the *pending* state until it is explicitly acknowledged with XACK.

Streams in this project

■ **`order_completed`**
Published by the **Payment API** (background task) after marking an order as *completed*. Contains the full order dict (product_id, quantity, …). Consumed by the **Inventory Consumer** to decrement stock.

■ **`refund_order`**
Published by the **Inventory Consumer** when it cannot find the product (deleted between order creation and completion). Contains the original order data. Consumed by the **Payment Consumer** to mark the order as *refunded*.

Key Redis Streaming commands used

Command	Purpose
`XADD stream * field value …`	Append a new message to the stream. Redis auto-generates the message ID.
`XGROUP CREATE stream group $ MKSTREAM`	Create a consumer group (called once at startup). `$` = only new messages.
`XREADGROUP GROUP g c STREAMS s >`	Read undelivered messages for this consumer group. `>` = never-delivered only.
`XACK stream group message-id`	Acknowledge a message. Without this, the message is redelivered forever.

5. End-to-End Order Flow

1. **User clicks "Place Order" in the React frontend**
   The frontend opens a modal showing quantity picker and a live price summary (subtotal + 20% fee + total).

2. **POST /orders → Payment API (port 8001)**
   Body: `{ "id": "", "quantity": 2 }`. The Payment API calls `GET http://localhost:8000/products/{pk}` on the Inventory API to get the current price.

3. **Order saved with status "pending" — response returned immediately**
   A **BackgroundTask** is queued (does not block the HTTP response). The frontend receives the order with `status: "pending"`.

4. **Background task runs (≈ 5 second simulated delay)**
   `order.status = "completed"` is saved to Redis. Then `redis.xadd("order_completed", order.model_dump())` publishes the event to the stream.

5. **Inventory Consumer reads the event from the stream**
   The consumer loop calls `XREADGROUP`, finds the message, looks up the product, and decrements `product.quantity -= order.quantity`. Finally it calls `redis.xack()` to remove it from the pending list. If the product no longer exists, it publishes to the `refund_order` stream instead.

6. **Frontend polls GET /orders/{pk} every 3 seconds**
   When the returned status changes from *pending* to *completed*, the status badge updates automatically. The product stock on the Products tab is also refreshed so the new quantity is visible.

6. React Frontend

The frontend is a single-page React application (Create React App) that talks directly to both backend services over HTTP. It uses `useState` and `useEffect` hooks for state management, and a `setInterval`-based polling loop to watch pending orders.

Key constants (App.js)

```
const INVENTORY = 'http://localhost:8000';  // Inventory API base URL
const PAYMENT   = 'http://localhost:8001';  // Payment API base URL
```

Features

Feature	How it works
Product listing	`GET /products` on mount; displays name, price, stock badge
Add product	Inline form → `POST /products` → list refreshes
Delete product	`DELETE /products/{id}` → list refreshes
Order modal	Quantity picker with live price breakdown (subtotal · fee · total)
Place order	`POST /orders` → order added to Orders tab
Auto-poll	Every 3 s, re-fetches each pending order; stops when all are settled
Stock refresh	Products re-fetched automatically when any order completes
Toast notifications	Success / error messages appear for 3 seconds then fade

7. How to Run the Project

Open **5 terminal windows** and run one command per window in this order:

#	Directory	Command	What it starts
1	`inventory/`	`uvicorn main:app --reload --port 8000`	Inventory REST API
2	`inventory/`	`python consumer.py`	Inventory Stream Consumer
3	`payement/`	`uvicorn main:app --reload --port 8001`	Payment REST API
4	`payement/`	`python consumer.py`	Payment Stream Consumer
5	`frontend/`	`npm start`	React dev server at :3000

Prerequisites

```
# Python dependencies (run inside each service folder)
pip install -r requirements.txt

# Node dependencies (run inside frontend/)
npm install

# Redis must be running (Docker example)
docker run -d -p 6379:6379 redis:alpine

# Each service folder needs a .env file
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_PASSWORD=
```